CODIZE

Product, Growth & Execution Plan

PREVENTION-FIRST, RECOVERY-CAPABLE EDITION | JULY-OCTOBER 2026

Teach students how to build with AI without giving up understanding or control.

Prevent the 80% Trap. Recover when it happens.

Current implementation focus: M15A -> M15B -> M15C -> M16A/B/C

PURPOSE OF THIS BRIEF

A single source of truth for Codize product positioning, beginner and recovery pathways, architecture, module sequencing, Fable priorities, beta validation, institutionalization, metrics, and Congressional App Challenge readiness. Use it to keep both Nolan and the Codize implementation chat aligned.

Version 2.0 - July 13, 2026. This edition supersedes the recovery-first brief and incorporates the prevention-first, recovery-capable product correction while preserving the provenance, uncertainty, source-traceability, and verification rules.

1. Executive Dashboard

> **NORTH-STAR THESIS**
>
> Codize teaches student builders how to use AI without giving up understanding or control. Beginners learn the workflow before the patch loop begins. Builders who are already stuck use the same system to regain control.

Workstream	State	Immediate implication
M14	**COMPLETE**	Artifact-Aware Project Defense is already built.
M15A	**CURRENT**	Implementation Import foundation: secure storage, APIs, validation, filtering, tests, and docs.
M15B-C	**NEXT**	Import UI, then a structured, editable, provenance-aware Change Map.
M16A-C	**MUST FOLLOW**	Reuse the confirmed Change Map through Review, Verification/Evidence, Defense, and Report.
M17	**REVISED**	Adaptive entry, Guided Build, and pilot readiness - after the shared core is stable.
Fable 5	**ENDS JUL 19**	Use remaining access for architecture, AI-heavy integration, security, reliability, and handoff.

The correction in one sentence

Codize is no longer framed mainly as a rescue tool for experienced AI builders. It is a guided AI-coding workflow for beginner and early-intermediate student builders, with an advanced recovery path for users already inside the 80% Trap.

What changes

- Primary audience broadens to beginner and early-intermediate student builders who want guidance before they lose control.
- The 80% Trap becomes both a failure Codize prevents and a failure Codize helps repair.
- M17 changes from mostly Pilot Readiness into Adaptive Entry, Guided Build, and Pilot Readiness.
- M18 becomes Learning Progress + Defense Report 2.0.
- Beta validation becomes dual-path: Guided Builders and Recovery Builders.
- Institutional pilots lead with learning to build with AI correctly, while retaining Project Recovery for advanced users.

What does not change

- Do not interrupt or restart M15A.
- M15B, M15C, and M16A/B/C remain the immediate technical priority.

- The confirmed Change Map remains the shared source for Review, Verification suggestions, Defense, and the Report.
- Provenance, source traceability, uncertainty, student judgment, and honest verification language remain permanent rules.
- The 80% Trap, Stop Debugging Blindly, Stay the Engineer, and the patch-loop story remain core brand assets.
- The College Zoom ladder, institutionalization objective, feature freeze, and CAC preparation sequence remain intact.

Non-negotiable priorities through July 19

[ ] Finish M15A cleanly. Do not interrupt or restart it.
[ ] Complete M15B and M15C before adding lower-priority features.
[ ] Complete the core of M16A/B/C, even if M17 must wait.
[ ] Preserve provenance, uncertainty, and source traceability in the data model.
[ ] End with tests, deployment, documentation, known limitations, rollback point, and a release tag.
[ ] Do not preserve a date by sacrificing sleep, security, or core coherence.

2. How to Use This Brief

Use this document as an operating manual, not as a rigid promise that every date must be hit perfectly. The order and exit criteria matter more than the exact day. When schedule pressure appears, preserve core product coherence before feature breadth.

> **SOURCE-OF-TRUTH RULE**
>
> The Codize implementation chat should inspect the repository before acting, reconcile this plan with completed work, and never duplicate a finished milestone. Current repository state outranks stale roadmap labels, but product principles in this brief should remain stable unless evidence justifies a change.

Table of contents

Section	Purpose
1. Executive Dashboard	Current state, strategic correction, and immediate priorities
2. How to Use This Brief	Operating rules and source-of-truth hierarchy
3. Strategic Product Correction	Prevention-first, recovery-capable direction
4. Positioning and Homepage	80% Trap, beginner bridge, and audience messaging
5. User Paths and Adaptive Support	Guided Build, Builder, and Recovery experiences
6. Trust and Claims Model	Provenance, uncertainty, evidence, and language
7. Core Architecture	Shared loop and Change Map integration
8. Module Roadmap	M15 through M21 with revised M17/M18
9. Fable Sprint	July 13-19 priorities and handoff
10. Discovery and Beta	Dual-path testing and product hypotheses
11. Adoption and Distribution	Recruitment, partners, social content, and growth loops
12. Institutionalization	College Zoom B- to A+ operationalized
13. Metrics and Evidence	What to measure and how to tell the iteration story
14. Master Timeline	July through October target dates
15. Congressional App Challenge	Feature freeze, submission package, and rules
16. Decision Rules	Scope control, prioritization, and schedule fallback
17. Master Checklist	Date-based on-track checklist
18. Codize Chat Handoff	Compact implementation instructions
19. Final Thesis and Sources	Canonical outcomes and verified references

3. Strategic Product Correction: Prevention-First, Recovery-Capable

> **DIAGNOSIS - NOT A FAILED DIRECTION**
>
> You did not build the wrong product. You built the advanced back half before fully designing the beginner front door. M14-M16 become the shared learning and recovery engine. The pivot is an expansion of the entry experience, not a rewrite of the core.

Old emphasis vs. revised direction

Dimension	Recovery-first emphasis	Revised direction
Primary user	Already built a substantial AI-assisted project and lost control	Wants to build with AI but needs structure; recovery users remain supported
Entry point	After the patch loop begins	Before the first prompt or after a project becomes difficult
80% Trap	Main entry condition	Named failure Codize prevents and helps repair
Primary value	Regain control	Learn to build with AI without losing control
Pilot requirement	Bring an existing AI-assisted project	Start a guided project or recover an existing one
Institutional pitch	Repair weak understanding after AI use	Teach a responsible, effective, explainable AI-building workflow

Audience model

Audience	What they are thinking	Codize response
Guided Beginner	"I want to build an app with AI, but I do not know where to start."	Guided scope, plain-language architecture, adaptive Prompt Builder, guided Review, testing, and Defense.
Early / Intermediate Builder	"AI helps me build, but I am not sure what I understand or what I should test."	Shared Build Loop with moderate scaffolding, Change Map, deliberate Review, Verification, and Defense.
Recovery Builder	"My project keeps breaking and I no longer know what changed."	Import existing work, reconstruct context, identify gaps, test behavior, and regain control.

One product, not three products

ADAPTIVE ENTRY

What brings you to Codize?

[Start my first AI-assisted project]

```
[ Build a new project with guidance ]
[ Regain control of an existing project ]

All paths -> Goal -> Plan -> Prompt -> Use AI -> Import -> Change Map
        -> Review -> Test + Evidence -> Defense -> Report
```

Scope boundary

Codize is not currently a complete programming-syntax curriculum. It should not try to become Codecademy, an AP Computer Science course, or a full language-learning platform before validation. Its educational domain is the AI-assisted project-building workflow: planning, prompting, understanding changes, reviewing, testing, recording evidence, explaining, and maintaining project ownership.

> **PERMANENT SCOPE TEST**
>
> Every major feature should strengthen at least one of: planning, prompting, understanding changes, reviewing, testing, evidence, explanation, or project ownership. If it does not, postpone it.

4. Canonical Positioning and Homepage Strategy

Brand hierarchy

Role	Canonical language	Job
Named problem	The 80% Trap	Makes the failure pattern memorable.
Hero	AI built your first 80%. Now you are stuck fixing the rest.	Creates recognition for users already in the patch loop.
Immediate pain / CTA	Stop Debugging Blindly	Names the practical frustration.
Identity promise	Stay the engineer when the project starts breaking.	Frames understanding and control as identity.
Product explanation	Plan. Prompt. Review. Verify. Explain. Defend.	Explains the missing workflow after attention is earned.
Campaign	Can you defend what AI built?	Creates a challenge, workshop, or social hook.
Beginner bridge	Build with AI without falling into the 80% Trap.	Invites users before failure occurs.
Recovery bridge	Already stuck? Bring your project back under control.	Keeps the advanced recovery path visible.

Revised role of the 80% Trap

Keep the 80% Trap. It is still Codize's strongest named problem. The change is conceptual: the 80% Trap is no longer the only reason a user enters Codize. It is the failure state that the beginner workflow prevents and the recovery workflow repairs.

> **RECOMMENDED HOMEPAGE BRIDGE**
>
> Starting your first AI build? Learn the workflow before the patch loop begins.
>
> Already stuck? Bring your project back under control.

Marketing sequence

MESSAGING FUNNEL

Pain / aspiration

-> "I want to build with AI" or "My AI-built project keeps breaking"

Recognition

-> "I do not know where to start" or "I lost track of what changed"

Desired outcome

-> Start confidently, stay in control, keep extending the project

Codize workflow

-> Plan, Prompt, Import, Change Map, Review, Test, Explain

Proof

-> Learning Progress + Defense Report

Audience-specific acquisition language

Audience	Primary message
Beginner	Want to build an app with AI but do not know where to start? Codize guides you from idea to prompt to working feature while teaching you what changed and how to stay in control.
Early builder	AI can generate the first version fast. Codize helps you understand the changes, test the behavior, and keep building without turning every bug into another blind patch.
Recovery user	Already stuck in patch loops? Bring the project back into Codize, reconstruct what changed, test what matters, and regain control.
Teacher / coach	Move beyond simply allowing or detecting AI use. Give students a workflow that teaches planning, review, testing, and explanation.
AI Vanguard / school initiative	Teach students how to build with AI without making understanding optional.

Marketing rules

- Do not lead with "responsible AI," "extra steps," educational friction, or a Defense Report.
- Do not accuse users of being lazy or tell them they do not understand their code.
- Lead beginner users with possibility and guidance; lead recovery users with patch loops, regressions, unread diffs, and lost control.
- Describe Codize as the missing workflow, not more homework.
- The Defense Report is proof of the process, not the only reason to begin.
- Prefer "AI-assisted code" when emphasizing human ownership.
- Do not claim Codize proves code is correct or independently verifies implementation correctness.

5. User Paths and Adaptive Support

Shared core journey

ONE SHARED LOOP

ENTRY

-> Goal

-> Plan

-> Prompt

-> Use an external AI coding tool

-> Bring Back What Changed

-> AI-generated Change Map draft

-> Student edits and confirms

-> Review decisions

-> Suggested verification checks

-> Student performs checks and records evidence

-> Artifact-Aware Project Defense

-> Learning Progress + Defense Report

Path A - Guided Build

1. Choose a meaningful but manageable project or starter template.
2. Identify who the project helps and reduce it to a realistic first version.
3. Learn the architecture in plain language only when each concept becomes relevant.
4. Build a scoped prompt with examples, constraints, no-touch zones, and explanations.
5. Use an external AI tool to generate the first implementation.
6. Import what changed and review an editable Change Map.
7. Use guided Review and Verification suggestions that explain why each area matters.
8. Answer implementation-specific Defense questions matched to project complexity and experience level.
9. Leave with visible learning progress and a Defense Report.

Path B - Builder Mode

Builder Mode uses the same core workflow with less explanation and more control. It is appropriate for students who know basic coding concepts and have already built small projects. Codize still provides structure but does not over-explain every term.

Path C - Recovery Mode

10. Import an existing AI-assisted project, AI response, diff, code, changed files, or a description.
11. Reconstruct a Change Map and preserve uncertainty where the import is incomplete.
12. Identify what the student can confirm, what remains unclear, and what needs inspection.
13. Review the most consequential changes rather than every line equally.
14. Generate verification suggestions from confirmed and uncertain areas.
15. Record actual test results and evidence.
16. Use Project Defense to expose gaps and rebuild a usable mental model.
17. Export a report that records what is known, what was tested, and what remains unresolved.

Adaptive dimensions

Dimension	Beginner	Intermediate	Advanced / Recovery
Terminology	Plain language first; technical term introduced with explanation	Technical term plus short context	Technical terminology by default
Project scope	Starter projects and strong scope limits	User-defined project with guardrails	Existing architecture or complex project
Prompt Builder	Examples, suggested wording, why each field matters	Moderate scaffolding	Minimal hand-holding and deeper constraints
Review	Explain why the changed area matters	Targeted review guidance	Architecture, security, and ripple-effect analysis
Verification	Concrete manual checks and expected observations	Behavior + edge-case suggestions	Deeper auth, data, failure-mode, and security checks
Defense	Foundation questions grounded in actual artifacts	Implementation decisions and system effects	Architecture tradeoffs, ripple effects, failure modes

Adaptive Defense examples

Level	Example grounded question
Beginner	Where does your app save tasks, and how do you know they remain after the page refreshes?
Intermediate	What connects each task to its owner, and where is that relationship stored?
Advanced	Where is ownership enforced, and what prevents one authenticated user from requesting another user's task?

> **IMPORTANT**
>
> Adaptive difficulty should change support and question depth, not make passing automatic. Every Defense question must remain grounded in student-confirmed project artifacts and the project's actual complexity.

6. Trust, Provenance, Uncertainty, and Verification Model

> **PERMANENT TRUST MODEL**
>
> Never merge student input, AI inference, and student confirmation into one ambiguous object. The entire product should preserve where a claim came from, what the AI inferred, what the student accepted or changed, what the student tested, and what evidence the student supplied.

Required provenance layers

PROVENANCE CHAIN

1. Student-provided source material
2. AI-generated Change Map draft
3. Student-edited and student-confirmed Change Map
4. Student-recorded Review decisions
5. Codize-suggested verification checks
6. Student-recorded verification results
7. Student-provided Evidence
8. Artifact-Aware Defense grounded in the record
9. Defense Report that preserves the distinctions

Source traceability

Every Change Map item should retain enough metadata to answer: "Why does this item appear?" The exact schema may change, but conceptually each item should preserve:

- Claim or extracted observation
- Origin: student-provided, AI-inferred, or student-created
- Source type: AI response, diff, code, file list, student description, or other artifact
- Source reference: file, section, line range, or artifact identifier when available
- Student status: confirmed, edited, rejected, uncertain, or needs inspection
- Timestamps and ownership context as appropriate

CONCEPTUAL ITEM - SCHEMA MAY DIFFER

```
{
  "claim": "Authentication middleware changed",
  "origin": "ai_inferred",
  "source_type": "git_diff",
  "source_reference": "middleware.ts lines 14-38",
  "student_status": "confirmed"
}
```

Uncertainty states

State	Meaning	Downstream behavior
Confirmed	Student agrees the item accurately represents the implementation	May ground Review, Verification suggestions, Defense, and Report.
Edited	Student changed the draft before confirming it	Use the edited student-confirmed version; retain draft history if practical.
Rejected	Student says the inference is wrong or irrelevant	Do not present it downstream as a project fact.
Uncertain	Student cannot yet determine whether it is correct	Surface for inspection and cautious questioning.
Needs inspection	The item requires looking at code, behavior, or another artifact	Create a concrete next action; do not force certainty.

Language rules

Situation	Acceptable language	Avoid
AI draft	"This appears to be what changed. Review and correct it."	"This is what changed."
Uncertain inference	"The draft suggests session handling may have changed. Did it?"	"Session handling changed."
After confirmation	"Your confirmed Change Map records a change to auth.py."	Presenting raw AI inference as confirmed.
Verification suggestion	"Because authentication changed, test login, logout, expiration, and unauthorized access."	Marking checks complete automatically.
Recorded result	"You recorded that login passed with this evidence."	"Codize verified authentication is correct."
Report	"Student-recorded verification result: Pass."	"Verified secure" unless independently established by a separate method.

Verification model

CORRECT VERIFICATION FLOW

Confirmed Change Map says authentication changed

-> Codize suggests login, logout, expiration, and unauthorized-access checks

-> Student performs the checks

-> Student records Pass / Fail / Skipped / N/A

-> Student adds evidence or explains why evidence is unavailable

-> Defense may ask about the result and its limitations

-> Report records the student's result without converting it into absolute proof

7. Core Product Architecture and Shared Data Flow

> **ARCHITECTURAL CENTER**
>
> The confirmed Change Map becomes the shared source for everything after implementation import. It reduces repeated work without replacing student judgment.

Required end-to-end journey

18. Student defines the project goal and who it helps.
19. Student chooses a guided, standard, or recovery entry path.
20. Student plans architecture at an experience-appropriate level.
21. Student builds a scoped prompt with constraints and no-touch zones.
22. Student uses an external AI coding tool.
23. Student imports useful implementation material.
24. Codize generates a structured, provenance-aware Change Map draft.
25. Student edits, rejects, marks uncertainty, and confirms items.
26. Review begins from the confirmed map rather than a blank page.
27. Codize suggests verification targets; the student performs checks.
28. Student records results and evidence.
29. Artifact-Aware Defense asks grounded, implementation-specific questions.
30. Learning Progress and Defense Report preserve the workflow record and unresolved risks.

Information reuse rule

Once the student confirms a Change Map item, Codize should reuse that information across Review, Verification suggestions, Defense, and the Report. Later steps should ask for judgment, action, evidence, or explanation - not duplicate transcription.

Captured once	Reused downstream	New human work still required
Changed files	Review, Verification suggestions, Defense, Report	Student decides whether changes are acceptable and explains why.
Behavior changes	Review, test suggestions, Report	Student performs tests and records outcomes.
Implementation decisions	Review, Defense, Report	Student evaluates tradeoffs and defends the decision.
Unresolved risks	Verification, Defense, next actions	Student inspects, tests, or marks remaining uncertainty.
Verification targets	Verification workflow and Report	Student performs checks; Codize never auto-completes them.

Definition of done for the shared core

SHARED CORE EXIT CRITERIA

Prompt

-> Implementation Import

-> AI-generated Change Map draft

-> Student confirmation / correction

-> Review decisions

-> Verification suggestions

-> Student-recorded results + Evidence

-> Artifact-Aware Project Defense

-> Learning Progress + Defense Report

All data persists across refresh, remains ownership-secured,
preserves provenance, and avoids repeated entry.

8. Detailed Module Roadmap

> **ROADMAP RULE**
>
> Finish the shared core before expanding the beginner front door. M15 and M16 serve every audience. The major product correction begins in M17, not by rewriting current work.

M15 - Bring Back What AI Changed

M15A - Implementation Import Foundation | CURRENT

[ ] Ownership-secured storage
[ ] API contracts
[ ] Imported AI response
[ ] Pasted diff or selected code
[ ] Changed-file metadata
[ ] Student description
[ ] Validation and input limits
[ ] Secret filtering
[ ] Missing-data behavior
[ ] Tests and documentation

Exit criterion: the student can safely save useful implementation context, retrieve only their own records, and continue after refresh without exposing secrets or corrupting project state.

M15B - Bring Back What AI Changed UI

Create a progressive interface titled: "What did AI change?" The student can provide any useful combination of:

- AI response
- Git diff
- Selected code
- Changed files
- Short description of the result

Do not make every field mandatory. The screen should feel like a practical handoff from the external AI tool, not a documentation assignment.

Exit criterion: a beginner can understand what to paste, an experienced user can provide richer artifacts, and incomplete input is handled honestly.

M15C - Structured, Editable, Provenance-Aware Change Map

Generate a student-editable draft containing:

- Changed files
- Behavior changes
- Major implementation decisions
- Out-of-scope modifications
- Security-sensitive areas
- Unresolved risks
- Unverified behavior
- Questions the student should understand
- Source references for each item
- Origin and student status for each item

The student must be able to confirm, edit, reject, mark uncertain, or mark an item as needing inspection.

Exit criterion: every downstream claim can be traced to source material and distinguished as AI-inferred versus student-confirmed.

M16 - Confirmed Change Map Workflow Integration

M16A - Confirmed Change Map -> Review

- Prefill Review from the confirmed Change Map.
- Student still chooses Accept, Reject, Edit, Uncertain, or Needs verification.
- Do not let AI review its own output and declare it acceptable.
- Do not ask the student to retype changed files or behavior already recorded.
- Preserve the relationship between each Review decision and its Change Map item.

Exit criterion: Review feels like deliberate decision-making, not another blank form.

M16B - Confirmed Change Map -> Verification Suggestions + Evidence

- Generate suggested checks from confirmed changes and explicitly uncertain areas.
- Adapt explanations and test depth to experience level later, without changing the underlying data model.
- Never mark a suggested check complete.
- Student performs the check and records Pass, Fail, Skipped, or N/A.
- Student attaches evidence or records why evidence is unavailable.
- Keep suggested checks distinct from student-recorded results.

Exit criterion: Codize guides testing without pretending it performed the test.

M16C - Confirmed Change Map -> Artifact-Aware Defense + Defense Report

Defense context priority:

- Student-confirmed Change Map information
- Student-recorded Review decisions
- Student-recorded Verification results
- Student-provided Evidence
- Raw imported material only as cautious supporting context

Defense questions should preserve uncertainty. AI-only inferences must be phrased as questions, not facts. The Report should preserve the same provenance distinctions.

Exit criterion: the gate asks project-specific questions grounded in the student's actual record, and the Report accurately represents what was provided, inferred, confirmed, tested, and evidenced.

M17 - Adaptive Entry, Guided Build, and Pilot Readiness

M17A - Intent Selection

- Ask why the user is entering Codize: first AI-assisted project, new guided project, or recovery of an existing project.
- A simplified public version may show: Start Building with AI / Fix an Existing AI Project.
- Store intent as user or project context without creating separate workflow architectures.

M17B - Experience Calibration

- Ask: New to building / I know basic coding concepts / I have built several projects.
- Use the response to adapt terminology, explanation depth, examples, project complexity, verification suggestions, and Defense question depth.
- Do not reduce standards into automatic passing.

M17C - Guided Project Start

- Choose a meaningful idea and identify who it helps.
- Reduce scope to a manageable first version.
- Explain architecture in plain language.
- Identify required components and features that should wait.
- Offer starter projects: study planner, task tracker, quiz app, habit tracker, or a simple site with one interactive feature.

M17D - Just-in-Time Teaching

Do not create long lessons before the student builds. Teach concepts when they become relevant. Use short explanations, examples, and an optional Learn More path.

> **EXAMPLE CONCEPT CARD**
>
> Why does this project need a database?
>
> Your task tracker needs somewhere to save tasks after the page closes. A database stores that information.

M17E - Adaptive Prompt Builder

- Beginners receive examples, suggested context, scope guidance, constraint examples, expected-output guidance, and explanations of why each section matters.
- Experienced users receive less hand-holding and more control.
- Preserve student decision-making; suggestions should not silently become final prompts.

M17F - Adaptive Review and Verification

- Beginner guidance explains why a changed area matters before suggesting what to inspect or test.
- Experienced guidance may use deeper technical checks and failure modes.
- The student still makes Review decisions and performs every recorded check.

M17G - Adaptive Project Defense

- Adapt question depth to project complexity and experience level.
- Keep every question implementation-specific and grounded in student-confirmed artifacts.
- Use progressive follow-ups rather than textbook trivia.
- Do not use experience level as a reason to ignore missing understanding.

M17H - Pilot Infrastructure

- Cohort codes: CODERSCHOOL-BETA, CROW-CS-2026, AI-VANGUARD-PILOT
- Funnel analytics from visit through report and return use
- Structured feedback at meaningful moments

- Quick-start choice between guided project and recovery project
- Private cohort association; no public student ranking

M17 exit criterion: a beginner can enter without an existing project, receive appropriate scaffolding, complete a small meaningful loop, and be measured as part of a pilot.

M18 - Learning Progress + Defense Report 2.0

Retain the full Defense Report and add a beginner-friendly learning layer.

- Project goal and who it helps
- Stack and architecture plan
- Final prompt and constraints
- Imported implementation material
- Changed files and student-confirmed Change Map
- Review decisions
- Student-recorded Verification results
- Student-provided Evidence
- Project Defense transcript and result
- Unresolved risks and next actions
- What the student can now explain
- What still needs review or testing

Exports: Markdown, print view, and copy. Add PDF only when reliable. Do not make the Report visually polished at the cost of inaccurate provenance.

M19 - Guided Starter Project + Judge / Teacher Demo

M19 now has two related experiences:

Experience	Purpose	Minimum content
Guided Starter Project	A real beginner completes the loop with support	Starter idea, scope, plain-language architecture, prompt, sample AI output, import, Change Map, Review, testing, Defense, learning progress.
Judge / Teacher Demo	Explains the full product in roughly two minutes	Pre-populated Study Planner with Login, an auth regression, Review decisions, test results, Defense, and final Report.

M20 - Reliability, Accessibility, and CAC Hardening

- Authentication reliability
- Loading, empty, and error states
- Structured API errors
- LLM timeout, retry, and fallback handling
- Stale-session recovery
- Responsive layout and browser testing
- Keyboard support and readable contrast
- Deployment health and monitoring
- Security review
- Source-code documentation
- AI-use disclosure
- Technical-decision records

Post-pilot - M21 Challenge Mode

Do not build Challenge Mode before validating the core learning and recovery loops. A challenge should reward meaningful process, not raw volume or LLM scores.

- Best-Defended Project
- Strongest Verification Process
- Best Human-AI Collaboration
- Most Thoughtful Technical Decision
- Most Improved Explanation

Avoid public raw gate scores, fastest-completion rankings, unlimited XP, and leaderboards based only on volume or LLM judgment.

9. Fable 5 Sprint: July 13-19

> **FABLE PRIORITY**
>
> Use Fable for difficult, cross-cutting, AI-heavy, security-sensitive work. Do not spend the remaining window on small copy changes, minor CSS, PDF polish, cohort-code UI, or broad beginner features while M15/M16 are incomplete.

Must complete while Fable is available

- M15A
- M15B
- M15C
- M16A core
- M16B core
- M16C core

Complete if time remains

- Architecture documentation
- Reliability review
- Pilot analytics foundation
- Handoff notes
- Known limitations
- Release tag and rollback point

Do not spend Fable time on yet

- Full M17 Guided Build
- Cohort-code UI polish
- Feedback-form UI
- PDF export
- Sample-project polish
- Marketing graphics
- Minor CSS

Day-by-day target

Date / window	Product focus	Validation / growth focus	Exit evidence
Jul 13-14	Finish M15A; begin M15B	No recruitment yet; prepare QA scenarios	Clean M15A commit, tests, migration/API docs
Jul 15	Complete M15B	Confirm beginner and advanced import wording is understandable	Working progressive import UI
Jul 16	Complete M15C	Test ambiguous and incomplete imports	Editable, traceable, uncertainty-aware Change Map
Jul 17	M16A + M16B core	Test no repeated data entry and honest verification language	Review integration and suggested-check pipeline
Jul 18	M16C core; end-to-end stabilization	Run full workflow on one sample project	Grounded Defense and accurate Report context

Date / window	Product focus	Validation / growth focus	Exit evidence
Jul 19	No new major module	Smoke test, deploy, document, hand off	Production build, known limitations, rollback, release tag

Hard fallback rule

> **WHEN THE SCHEDULE SLIPS**
>
> If M16 is not stable by July 18, drop pilot-feature implementation and finish M16 properly. Do not rush M17 into the Fable window. A coherent shared core is more valuable than an incomplete beginner front door.

July 19 handoff package

[ ] Architecture overview
[ ] Implementation Import data/API notes
[ ] Change Map schema, provenance, and uncertainty model
[ ] LLM extraction pipeline and fallback behavior
[ ] Security and ownership boundaries
[ ] Deployment procedure
[ ] Production smoke-test checklist
[ ] Known limitations
[ ] Post-Fable next milestones
[ ] Rollback point and clean release tag

10. Discovery and Dual-Path Beta Validation

What the Instagram result means

One unanswered Instagram story does not prove the recovery market is too small. It does show that the current call-to-action may require experience your immediate network does not have, may use language people do not identify with, or may make people feel unqualified to respond. Treat it as a signal to broaden discovery, not as proof to delete the recovery path.

July 20-23 - Founder QA + discovery

Run the full M15/M16 workflow with yourself and 2-4 trusted people. Also conduct 3-5 short discovery conversations.

- Have you wanted to build something with AI? What stopped you?
- What would you want to build?
- Which part feels most confusing: idea, scope, prompting, code, testing, or fixing?
- Have you used AI for code without knowing what changed?
- Would you rather start from a guided project or bring your own?
- What outcome would make a 30-60 minute Codize session worth it?

July 24-28 - Minimum M17 entry layer

- Choose path
- Choose experience level
- One guided starter project
- Plain-language architecture guidance
- Adaptive copy in Prompt Builder, Review, Verification, and Defense where feasible
- Basic cohort/experiment labeling if low risk

Do not build the entire long-term curriculum. Build the smallest front door that lets a beginner complete the existing shared core.

July 29-August 7 - Dual-path beta

Cohort	Target	Experience	What they do
A. Guided Builders	6-10 students	Want to build with AI; limited project experience	Complete one guided starter project through Prompt, Import, Change Map, Review, testing, Defense, and Report.
B. Recovery Builders	3-5 students	Have an existing AI-assisted project or patch-loop experience	Import existing work, reconstruct context, review, test, defend, and record unresolved risks.

Core product hypotheses

Hypothesis	How to test	Evidence that supports it
H1: Guided entry recruits more students than recovery-only messaging	Compare response, signup, and session attendance	Higher qualified response and lower intimidation without worse completion.
H2: A confirmed Change Map reduces form fatigue	Observe repeated entry and ask which steps feel redundant	Users reuse captured data and report less repeated work.
H3: Just-in-time teaching is more usable than front-loaded lessons	Compare engagement and confusion during one starter project	Students continue building and can explain concepts when asked.

Hypothesis	How to test	Evidence that supports it
H4: Project Defense reveals real understanding gaps	Observe whether questions expose missing project-specific knowledge	Students identify a gap, revise an explanation, or return to inspect implementation.
H5: The Report creates a meaningful payoff	Ask whether users save, share, or value the artifact	Users describe it as useful for demos, interviews, teachers, or future work.

Guided Builder metrics

- Recruitment response and session attendance
- Time to choose a project and reach a scoped first version
- Prompt Builder completion
- Time to first useful insight
- Ability to explain what AI changed
- Review and Verification completion
- Project Defense completion
- Would build another project with Codize
- Could explain at least one concept they did not understand at the start

Recovery Builder metrics

- Could provide enough implementation context to create a useful Change Map
- Codize revealed a missing understanding or untested behavior
- User corrected at least one AI inference
- User completed a meaningful Review and test cycle
- User felt more confident extending or explaining the project
- Would use Codize for the next phase or project

Observation protocol

For the first users, scheduled sessions are more valuable than sending a link. Watch quietly, note where they hesitate, and ask questions afterward. Do not coach them through every confusing screen; confusion is product evidence.

- Where did the user hesitate?
- Which words were unclear?
- Which step felt like homework?
- What did they try to skip?
- What made them feel progress?
- Did the Change Map feel useful enough to correct?
- Were verification suggestions actionable?
- Did Defense ask something genuinely revealing?
- Would they use Codize without you sitting beside them?

11. Adoption, Distribution, and Growth Loops

> **ADOPTION PRINCIPLE**
>
> Students do not need to be motivated by abstract responsibility. Give them an immediate desired outcome: build something they care about, stop getting lost, keep extending it, and leave able to explain what they made.

Recruitment messages

Use case	Message
Guided workshop	Want to build an app with AI but do not know where to start? Codize walks you from idea to prompt to working feature while teaching you what changed and how to stay in control.
Recovery session	Already built with AI and stuck in patch loops? Bring your project into Codize, reconstruct what changed, test what matters, and regain control.
General beta	Help test a new way for students to build with AI without giving up understanding. You can start a guided project or bring one you already made.
Teacher / coach	Codize gives students a structured process for planning, prompting, reviewing, testing, and explaining AI-assisted work.
Challenge event	Build with AI without falling into the 80% Trap. Then see whether you can defend what you built.

First distribution order

31. Warm guided beta: friends, classmates, hackathon peers, and students who are curious about AI building - not only people who already built an app.
32. TheCoderSchool pilot: a guided workshop for intermediate students, with Recovery available for those who bring existing projects.
33. AI Vanguard partnership: recruit students across schools around AI literacy and student voice.
34. Mr. Crow review: validate both the Guided Build workflow and the quality of Project Defense.
35. High-school pilot: small optional cohort before any schoolwide ask.
36. Second organization: repeat the protocol with comparable measures.

TheCoderSchool pilot

> **RECOMMENDED WORKSHOP**
>
> Build Your First AI App Without Losing Control
>
> - Choose a small project.
> - Scope the first version.
> - Build a prompt.
> - Use AI to generate one feature.
> - Bring back what changed.
> - Review and test it.
> - Explain the implementation.

Ask for a small pilot, not branch-wide adoption. A realistic first cohort is 5-10 students with enough coding comfort to use a starter project. The owner/coach can provide feedback on fit, pacing, and whether the workflow complements existing instruction.

AI Vanguard partnership

AI Vanguard may be more valuable as a distribution and pilot partner than only as a board title. A partnership could recruit students from multiple schools, frame Codize as a concrete responsible-AI practice, and produce anonymous aggregate findings about where students struggle.

> **POSSIBLE EVENT**
>
> Build with AI Without Falling into the 80% Trap

Mr. Crow

Approach Mr. Crow first as an expert reviewer, not as a distribution channel. Ask two questions:

37. Does Guided Build teach an appropriate AI-assisted coding workflow for students?
38. Does Project Defense distinguish project-specific understanding from generic technical knowledge?

Then share measured beta evidence and ask for a small optional pilot. This creates mentorship and validation before scale.

Social media

Reserve the Codize account and document the problem, the build, and real iteration. Social media should amplify proven value, not substitute for direct pilot recruitment.

Content pillar	Example
Possibility	"Want to build an app with AI but have no idea where to start?"
80% Trap recognition	Patch-loop terminal, unread diff, "You are negotiating with a codebase you never learned."
Can you defend this?	Show a small implementation decision and ask a grounded question.
Product iteration	"Beginners got stuck here, so we changed the workflow."
User outcome	With permission: what a student learned, tested, or realized.

Growth loops

Loop	Mechanism
Learning / Report loop	Student completes a build -> sees what they can explain -> saves or shares report/card -> friend becomes curious.
Workshop loop	Partner runs a guided session -> receives a pilot summary -> repeats with another cohort.
Teacher loop	Teacher reviews aggregate gaps -> uses Codize again -> shares protocol with another teacher.
Challenge loop - later	School or cohort completes a time-limited challenge -> projects receive human-reviewed recognition -> another group joins.
Iteration loop	Tester reveals friction -> Codize changes -> the change is documented publicly -> new users test the improved version.

12. Institutionalization and the College Zoom Ladder

Do not confuse labels with evidence

The goal is not to claim A+ because Codize was mentioned to multiple schools. The strongest proof is a working system, measured use, a repeatable pilot, and successful replication. One strong first pilot plus one real second implementation is more valuable than many outreach emails.

College Zoom level	Codize evidence required	Target window
B- - Analyze and hone	First attempt, real feedback, major retool. Already substantially completed.	Completed / current
B+ - Release and measure	Release v2, run dual-path beta, measure behavior, document at least one evidence-driven product change.	Late July-August 14
A- - Create scalable protocols	First institutional pilot, facilitator materials, sponsor, written results, and a process someone else can run.	August-September 14
A+ - Institutionalize laterally	Revise after institution one, replicate with a second organization, collect comparable evidence, reduce dependence on Nolan.	September 15-28 and beyond

Institutional pilot kit

[ ] Teacher / Coach Overview
[ ] Student Quick Start
[ ] Guided Build workshop curriculum
[ ] Recovery workshop option
[ ] Facilitator Script
[ ] Starter Project and Demo Project
[ ] Feedback Survey
[ ] Troubleshooting Guide
[ ] Privacy and Data Overview
[ ] Pilot Analytics / Results Template
[ ] Adoption Checklist
[ ] Known Limitations and Support Path

A- exit criteria

- One organization officially agrees to test Codize.
- An adult sponsor, teacher, coach, or organization leader is involved.
- A defined cohort completes a structured pilot.
- The same lesson, facilitator script, and feedback process can be reused.
- A written pilot report exists.
- Someone other than Nolan can facilitate the next session using the materials.

A+ exit criteria

- Institution one completed a pilot and produced actionable evidence.
- The protocol was revised based on that evidence.
- A second organization used substantially the same protocol.
- Comparable activation, completion, learning, and feedback measures were collected.
- A teacher, coach, or student leader helped run the second implementation.
- Codize no longer depends entirely on Nolan personally guiding every user.

13. Metrics, Evidence, and Iteration Stories

North-star evidence

WHAT SUCCESS SHOULD PROVE

Codize helps students start or recover an AI-assisted project while preserving understanding and control. The strongest evidence combines behavior, student explanation, recorded testing, and repeat use - not just signups.

Category	Core measures
Acquisition	Invited, reached, responded, registered, attended a session
Path selection	Guided Build, standard Build, Recovery
Activation	Scoped project, completed Prompt Builder, imported implementation, confirmed Change Map
Workflow	Review completed, verification checks performed, evidence recorded, Defense completed, Report generated
Learning / usefulness	New concept explained, understanding gap revealed, untested behavior identified, changed Review behavior
Retention	Returned within 7 days, began another phase or project, reused Codize without direct facilitation
Institutional	Sponsor satisfaction, willingness to repeat, facilitator independence, protocol reuse, second-site replication

Compare pathways

Metric	Guided Build	Recovery
Recruitment response	Does curiosity about building produce more responses?	Can experienced users be identified and recruited?
Time to first value	How quickly can a student scope a project or understand a concept?	How quickly does the Change Map reveal useful context?
Completion	Can a beginner finish a small loop?	Can a user recover enough context to complete Review/Test/Defense?
Understanding	Can the student explain what changed and why?	Did Codize expose a gap or rebuild a mental model?
Repeat intent	Would the student build another project with Codize?	Would the student use Codize on the next phase or project?

Evidence story format

ITERATION CHAIN

Tester evidence

-> specific problem observed

-> product change made

-> measured or observed result

```
-> next decision
```

Example: "Six of ten Guided Builders did not know what to paste into Implementation Import. We replaced the single large text box with examples and progressive input choices. In the next five sessions, four users completed import without facilitator intervention."

Do not rely on vanity metrics

- Signups without activation
- Page views without workflow completion
- Raw user count without target-user fit
- A large social following without meaningful use
- Public gate scores without demonstrated validity
- Plans to expand without a proven pilot
- Institution names mentioned without an actual cohort or repeatable protocol

14. Master Timeline: July-October 2026

> **TIMELINE RULE**
>
> Treat dates as target windows. Preserve sequence and exit criteria. A few days of slippage are acceptable; unstable architecture, misleading claims, or rushed pilots are not.

Date / window	Product focus	Validation / growth focus	Exit evidence
Jul 13-19	Finish M15A-C and M16A-C; tests, deploy, docs	No broad beta; prepare discovery and QA	Stable shared core and Fable handoff
Jul 20-23	Founder QA; fix blockers	3-5 discovery conversations	Known blockers, user-language insights, revised QA list
Jul 24-28	Minimum M17 entry layer	Recruit Guided and Recovery cohorts	Path selection, experience calibration, one starter project
Jul 29-Aug 7	Support beta issues; avoid feature sprawl	6-10 Guided + 3-5 Recovery testers	Comparable funnel, completion, learning, and feedback data
Aug 8-14	Retool into v2.1	Prioritize by frequency x severity x mission alignment	Documented iteration chain and improved build
Aug 15-24	Pilot-ready fixes and facilitator materials	First guided institutional pilot: TheCoderSchool or AI Vanguard	Sponsor feedback, pilot results, repeatability gaps
Late Aug-Sep 7	Refine Guided Build and Defense	Mr. Crow review; small high-school pilot ask	Educational/technical validation and school pilot plan
Sep 1-14	Build complete institutional kit	Run or complete first high-school pilot	A- evidence: protocol, sponsor, results, facilitator materials
Sep 15-28	Revise and stabilize protocol	Second organization / lateral replication	Beginning A+ evidence: comparable second-site use
Sep 29-Oct 5	Feature freeze; M18-M20 polish	No new broad outreach that threatens stability	Reliable Report, demo mode, accessibility, docs
Oct 6-18	CAC package and final hardening	Record demo, finalize disclosure and submission	Internal submission target: Oct 18
Oct 19-26	Buffer only; emergency fixes	Confirm submission and district requirements	Official deadline: Oct 26, 12:00 p.m. ET

What should be true at each major checkpoint

Checkpoint	Minimum truth
July 19	The shared core works end to end; provenance and verification language are trustworthy; a new model can continue from documentation.
July 28	A beginner can enter through a minimal guided path without bringing an existing project.
August 7	Both Guided and Recovery paths have real usage evidence.
August 14	At least one meaningful iteration is documented from evidence.

Checkpoint	Minimum truth
August 24	One organization has run or committed to a guided pilot.
September 14	A repeatable institutional kit and first institutional evidence exist.
September 28	A second organization has used or formally begun the same protocol.
October 5	No major features remain; the product is stable and demoable.
October 18	CAC submission package is complete and preferably submitted.

15. Congressional App Challenge Plan

> **OFFICIAL DEADLINE**
>
> October 26, 2026 at 12:00 p.m. Eastern. Keep October 18 as the internal target so late October can remain a buffer rather than a development sprint.

September 29-October 5 - Feature freeze

- No major new product expansion
- Learning Progress and Defense Report polish
- Guided starter and judge/teacher demo mode
- Reliability and error handling
- Accessibility and responsive behavior
- Performance and deployment health
- Documentation and source-code readiness

October 6-18 - Submission package

[ ] 1-3 minute demo video
[ ] Demo account or clearly accessible demonstration path
[ ] Guided starter or sample project
[ ] Architecture diagram
[ ] README and setup documentation
[ ] Technical decisions and tradeoffs
[ ] AI-use disclosure
[ ] Screenshots
[ ] Submission responses
[ ] Source-code readiness
[ ] Final end-to-end smoke test

CAC credibility rules

- Document the AI tools used and the specific ways they contributed.
- Document Nolan's problem selection, product thesis, architecture decisions, testing, evaluation, security review, and iterative direction.
- Be able to explain the implementation and tradeoffs without relying on generated descriptions.
- Prefer a stable, coherent, demoable core over many unfinished features.
- Use the demo to show both the beginner opportunity and the differentiated Change Map -> Review -> Test -> Defense loop.
- Do not present student-recorded tests as independent proof of correctness.

Recommended demo story

39. Open with the 80% Trap and the beginner problem: AI makes it easy to start, but not easy to stay in control.
40. Show the two entry paths: Start Building with AI / Fix an Existing AI Project.
41. Use the Guided Starter to show goal, scope, Prompt Builder, and plain-language architecture.
42. Show Implementation Import and the provenance-aware Change Map.
43. Show student Review, suggested checks, student-recorded results, and Evidence.
44. Show Artifact-Aware Project Defense adapting to the actual project.
45. End with Learning Progress + Defense Report and the institutional potential.

16. Decision Rules, Scope Control, and Guardrails

Feature gate

Before adding a major feature, ask:

46. Does this help a student plan, prompt, understand changes, review, test, record evidence, explain, or maintain ownership?
47. Does it improve recruitment, activation, completion, learning, or repeatability for the current validation stage?
48. Can it be built without weakening M15/M16 integrity, trust, or reliability?
49. Do user or pilot data support it, or is it mainly exciting to build?
50. Can the effect be measured?

If most answers are no, postpone the feature.

Priority formula after beta

RETOOLING RULE

Priority = Frequency x Severity x Mission Alignment

Then apply feasibility and risk as constraints.

Do not implement every request.

What not to build before validation

- Public individual leaderboards
- Public LLM gate-score rankings
- Social feed
- Marketplace or payments
- Large teacher dashboard
- Teams / collaboration suite
- GitHub OAuth
- VS Code extension
- Complex gamification
- Full programming-syntax curriculum
- Multiple-project support unless required by the current workflow

When behind schedule

- Protect M15/M16 data integrity and end-to-end coherence.
- Drop lower-priority M17 polish before dropping core integration.
- Drop visual polish before dropping ownership, security, and provenance tests.
- Do not start a new module on July 19.
- Document known limitations honestly rather than hiding them.
- Move a target date rather than using repeated all-nighters as the operating model.

Permanent product constraints

- Never present AI inference as confirmed implementation fact.
- Never silently convert student-recorded verification into Codize-verified correctness.
- Never make the student retype captured information merely to satisfy a form.
- Never let AI replace the student's Review judgment or verification action.
- Never let adaptive support turn Project Defense into automatic passing.
- Never let beginner expansion turn Codize into a generic programming course without evidence.

- Never optimize public competition around raw LLM scores or completion speed.
- Never sacrifice user trust for a cleaner-looking but misleading Report.

17. Master On-Track Checklist

Now through July 19

[ ] M15A complete, tested, documented, and committed
[ ] M15B usable with progressive optional inputs
[ ] M15C editable, provenance-aware, traceable, and uncertainty-aware
[ ] M16A Review integration working
[ ] M16B suggestions separated from student-recorded results
[ ] M16C Defense and Report grounded in confirmed records
[ ] Full end-to-end smoke test passes
[ ] Production deploy succeeds
[ ] Known limitations documented
[ ] Rollback point and release tag created
[ ] Post-Fable handoff package complete

By July 23

[ ] Founder QA completed with 2-4 trusted people
[ ] 3-5 discovery conversations completed
[ ] Top beginner barriers identified
[ ] Top recovery barriers identified
[ ] M17 minimum scope frozen

By July 28

[ ] Intent selection works
[ ] Experience calibration works
[ ] One guided starter project exists
[ ] Plain-language architecture guidance exists
[ ] Beginner can reach the shared loop without an existing project

By August 7

[ ] 6-10 Guided Builders recruited
[ ] 3-5 Recovery Builders recruited
[ ] Path, activation, completion, learning, and return-intent data collected
[ ] Observed friction documented
[ ] At least one session completed without Nolan guiding every click

By August 14

[ ] Top problems prioritized by frequency x severity x mission alignment
[ ] v2.1 changes implemented
[ ] At least one evidence -> problem -> change -> result chain documented
[ ] Institutional pilot package draft exists

By August 24

[ ] TheCoderSchool or AI Vanguard guided pilot completed or formally scheduled
[ ] Adult/organization sponsor identified
[ ] Pilot results or pre-pilot baseline documented
[ ] Facilitator materials revised

By September 14

[ ] Mr. Crow has reviewed the Guided Build and Project Defense approach
[ ] First high-school or equivalent institutional pilot completed or actively underway
[ ] Teacher/Coach Overview complete
[ ] Student Quick Start complete
[ ] Pilot Curriculum and Facilitator Script complete
[ ] Privacy/Data Overview complete
[ ] Written pilot report complete

By September 28

[ ] Second organization uses substantially the same protocol
[ ] Comparable evidence collected
[ ] Protocol revised after first implementation
[ ] Another facilitator can run the session with limited Nolan support

By October 5

[ ] Feature freeze active
[ ] Guided starter and judge demo stable
[ ] Learning Progress + Defense Report polished
[ ] Accessibility and reliability checks complete
[ ] AI-use disclosure and technical decisions documented

By October 18

[ ] 1-3 minute video complete
[ ] Demo path tested
[ ] Architecture diagram complete
[ ] README and submission responses complete
[ ] Source-code readiness confirmed
[ ] Final smoke test complete
[ ] Submission completed if possible

18. Compact Handoff for the Codize Implementation Chat

> **INSTRUCTION**
>
> Use the following as the compact current strategy. The full PDF remains the source of truth for detail. Inspect the repository before implementation and reconcile milestone names with actual completed work.

COPY / REFERENCE BLOCK

CODIZE STRATEGIC STATE - JULY 2026

Direction:
Prevention-first, recovery-capable.
Codize teaches students how to build with AI without giving up understanding or control. The same system helps users recover after they fall into the 80% Trap.

Do not interrupt current work:
M15A -> M15B -> M15C -> M16A -> M16B -> M16C

Shared architecture:
Student source -> AI Change Map draft -> student confirmation -> Review -> suggested checks -> student-recorded results + Evidence -> Artifact-Aware Defense -> Learning Progress + Defense Report

Permanent trust rules:
Preserve student-provided, AI-inferred, student-confirmed, student-tested, and student-evidenced information separately.
Keep source traceability and uncertainty states.
Never claim Codize independently verified correctness.
Never auto-complete verification or replace student Review judgment.

After M16:
M17 = Adaptive Entry + Guided Build + Pilot Readiness
M18 = Learning Progress + Defense Report 2.0
M19 = Guided Starter + Judge/Teacher Demo
M20 = Reliability, Accessibility, CAC Hardening

Primary audience:
Beginner and early-intermediate student builders who want structure.
Secondary audience:

Experienced builders who need to regain control.

Do not turn Codize into a full programming curriculum.
Do not build leaderboards, social features, or broad integrations
before the shared core and dual-path beta are validated.

Current implementation priority

Priority	Action
0	Finish the current milestone cleanly: tests, migration, documentation, commit.
1	Complete M15A-M15C.
2	Complete M16A-M16C core integration.
3	End Fable with reliability, documentation, known limitations, rollback, and release tag.
4	After July 19, implement the minimum M17 beginner front door.
5	Run dual-path beta before deciding how much more beginner or recovery emphasis to build.

19. Final Product Thesis and Verified Sources

> **CANONICAL PRODUCT THESIS**
>
> Codize teaches student builders how to use AI without giving up understanding or control.

Expanded thesis

AI makes it easier than ever to start building. Codize guides students through planning, prompting, reviewing, testing, and explaining what they create - whether they are beginning their first AI-assisted project or recovering one that has already fallen into the 80% Trap.

Beginner transformation

> **GUIDED BUILDER**
>
> "I had an idea. Codize taught me how to scope it, prompt AI, understand what changed, test the behavior, and explain what I built."

Recovery transformation

> **RECOVERY BUILDER**
>
> "My project had become a patch loop. Codize helped me reconstruct what changed, review the decisions, test the behavior, record evidence, and regain control."

Shared final outcome

THE CODIZE OUTCOME

I know what changed.

I reviewed the decisions.

I tested the behavior.

I recorded the evidence.

I can defend what I built.

Official competition references

2026 Congressional App Challenge Rules: official rules PDF

2026 Student Registration and Deadline: official registration page

Verified July 13, 2026: the official deadline is October 26, 2026 at 12:00 p.m. Eastern.

Homepage research citation note

The homepage already correctly labels the 80% Trap as Codize's name for a real pattern rather than a measured statistic. Keep that disclaimer. For the security section, continue citing the underlying research precisely and avoid generalizing beyond the studies.

Perry et al., "Do Users Write More Insecure Code with AI Assistants?" - ACM Digital Library

Fu et al., "Security Weaknesses of Copilot-Generated Code in GitHub Projects" - arXiv preprint

Recommended label for Fu et al.: "preprint 2023; ACM TOSEM 2025."

Document status

This edition is the active one-stop operating brief. It supersedes the prior recovery-first version while retaining its strongest architecture, trust model, scaling ladder, and Congressional App Challenge plan.